SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

SIMATS ENGINEERING - CSA17 ARTIFICIAL INTELLIGENCE LAB

Day 1 - Program 1	Course Outcome: CO1	Language: Python 3

Question

Write the python program to solve 8-Puzzle problem

Aim

To solve the 8-Puzzle problem using the A* search algorithm.

Algorithm / Explanation

The puzzle has a 3x3 board with tiles 1-8 and a blank (0). The goal is 1 2 3 / 4 5 6 / 7 8 _.

A* expands the state with the lowest f(n) = g(n) + h(n), where g(n) is the number of moves made and h(n) is the sum of Manhattan distances of all tiles from their goal positions.

A puzzle is solvable only if the number of inversions in the tile sequence is even; this is checked before searching.

Program

```
# 8-Puzzle Problem using A* search (Manhattan distance heuristic)
import heapq
from itertools import count

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)          # 0 represents the blank

def manhattan(state):
    dist = 0
    for i, v in enumerate(state):
        if v:
            gi = v - 1
            dist += abs(i // 3 - gi // 3) + abs(i % 3 - gi % 3)
    return dist

def neighbors(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    for dr, dc, move in ((-1, 0, "Up"), (1, 0, "Down"), (0, -1, "Left"), (0, 1, "Right")):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            n = nr * 3 + nc
            lst = list(state)
            lst[z], lst[n] = lst[n], lst[z]
            yield tuple(lst), move

def is_solvable(state):
    tiles = [x for x in state if x]
    inversions = sum(1 for i in range(8) for j in range(i + 1, 8) if tiles[i] > tiles[j])
    return inversions % 2 == 0

def solve(start):
    tie = count()
    pq = [(manhattan(start), 0, next(tie), start)]
    parent = {start: (None, None)}
    best_g = {start: 0}
    while pq:
        f, g, _, cur = heapq.heappop(pq)
        if cur == GOAL:
            path = []
            while parent[cur][0] is not None:
                prev, mv = parent[cur]
                path.append((mv, cur))
                cur = prev
            return path[::-1]
```

```
        for nxt, mv in neighbors(cur):
            ng = g + 1
            if nxt not in best_g or ng < best_g[nxt]:
                best_g[nxt] = ng
                parent[nxt] = (cur, mv)
                heapq.heappush(pq, (ng + manhattan(nxt), ng, next(tie), nxt))
    return None

def show(state):
    for i in range(0, 9, 3):
        print(" ".join(str(x) if x else "_" for x in state[i:i + 3]))
    print()

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
print("Start state:")
show(start)
if not is_solvable(start):
    print("This puzzle is not solvable")
else:
    solution = solve(start)
    print("Solved in", len(solution), "moves:", [m for m, _ in solution], "\n")
    for step, (mv, st) in enumerate(solution, 1):
        print(f"Step {step}: {mv}")
        show(st)
```

Output

```
Start state:
1 2 3
4 _ 6
7 5 8

Solved in 2 moves: ['Down', 'Right']

Step 1: Down
1 2 3
4 5 6
7 _ 8

Step 2: Right
1 2 3
4 5 6
7 8 _
```

Result

Thus the Python program was executed successfully and the output was verified.